

Notebook 11: Controlled Ablation Studies & Experimental Robustness Analysis

Project: Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms

1. Overview & Experimental Design

Ablation and stress testing isolate the exact contribution of each architectural innovation:

1.1 Controlled Component Ablations

We evaluate five controlled model variants against the complete Proposed **LAC-SSM**:

1. **Full Proposed Model (LAC-SSM)**: All modules active (Lead Embeddings + Dynamic Attention Masking + S4D State-Space Core + Lead Dropout + Heteroscedastic Uncertainty).
2. **Variant A: w/o Anatomical Lead Embeddings (E_{lead})**: Verifies the importance of anatomical lead identities.
3. **Variant B: w/o Dynamic Lead Masking**: Inputting zeroed signals without attention gating to prove the utility of explicit mask conditioning.
4. **Variant C: w/o State-Space Core (Replaced by Standard Conv/Dense)**: Evaluates the temporal representation capacity of S4D against standard 1D convolutions.
5. **Variant D: w/o Stochastic Lead Dropout Augmentation**: Highlights how lead dropout prevents overfitting to full 12-lead configurations.
6. **Variant E: w/o Heteroscedastic Uncertainty Gating**: Standard deterministic BCE loss.

1.2 Incomplete Lead Graceful Degradation Benchmark

Evaluating performance across available lead counts $k \in \{12, 11, 10, 9, 8, 6, 4, 3, 2, 1\}$ comparing:

- **Proposed LAC-SSM**
- **ResNet-1D Baseline**
- **1D Transformer Baseline**

1.3 Signal Noise & Clinical Artifact Robustness

Assessing diagnostic retention under:

- Additive Gaussian White Noise across SNR levels (30 dB down to 5 dB).
- Low-frequency respiratory baseline wander (0.2 Hz).
- High-frequency powerline interference (50 Hz).

```
In [1]: import os
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torch.nn as nn
import torch.nn.functional as F

from sklearn.metrics import f1_score, roc_auc_score, average_precision_score, hammi

# High-resolution styling
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'seaborn-v0_8-dark')
plt.rcParams.update({
    'font.sans-serif': 'DejaVu Sans',
    'font.size': 11,
    'axes.labelsize': 12,
    'axes.titlesize': 13,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'figure.titlesize': 14,
    'figure.dpi': 300
})

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Ablation & Robustness module running on device: {device}")

RESULTS_DIR = r"C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Mo
FIG_DIR = os.path.join(RESULTS_DIR, "figures")
TAB_DIR = os.path.join(RESULTS_DIR, "tables")
MET_DIR = os.path.join(RESULTS_DIR, "metrics")
ABL_DIR = os.path.join(RESULTS_DIR, "ablations")
ROB_DIR = os.path.join(RESULTS_DIR, "robustness")

for d in [FIG_DIR, TAB_DIR, MET_DIR, ABL_DIR, ROB_DIR]:
    os.makedirs(d, exist_ok=True)
```

Ablation & Robustness module running on device: cpu

```
In [2]: data = np.load(os.path.join(MET_DIR, "preprocessed_data.npz"))
X_test = data['X_test'] # (N_te, 12, 1000)
y_test = data['y_test'] # (N_te, 9)
target_classes = list(data['target_classes'])

inc_masks_data = np.load(os.path.join(MET_DIR, "incomplete_lead_masks.npz"))

class S4DLayer(nn.Module):
    def __init__(self, d_model=64, d_state=16):
        super().__init__()
```

```

self.d_model = d_model
self.d_state = d_state
self.log_A_real = nn.Parameter(torch.log(0.5 * torch.ones(d_model, d_state)
self.A_imag = nn.Parameter(math.pi * torch.arange(d_state).unsqueeze(0).rep
self.B = nn.Parameter(torch.randn(d_model, d_state) * 0.02)
self.C = nn.Parameter(torch.randn(d_model, d_state) * 0.02)
self.D = nn.Parameter(torch.ones(d_model))
self.log_step = nn.Parameter(torch.log(torch.tensor(0.01)))

def forward(self, u):
    B, T, H = u.shape
    step = torch.clamp(torch.exp(self.log_step), min=1e-4, max=0.05)
    A_real = -F.softplus(self.log_A_real) - 1e-4
    A = A_real + 1j * self.A_imag
    A_bar = torch.exp(A * step)
    B_bar = ((A_bar - 1.0) / (A + 1e-7)) * self.B
    t_steps = torch.arange(T, device=u.device, dtype=torch.float32)
    A_powers = A_bar.unsqueeze(-1) ** t_steps.unsqueeze(0).unsqueeze(0)
    kernel = 2.0 * torch.sum(self.C.unsqueeze(-1) * A_powers * B_bar.unsqueeze(
    u_conv = u.transpose(1, 2)
    kernel_weight = kernel.unsqueeze(1)
    y_conv = F.conv1d(F.pad(u_conv, (T - 1, 0)), kernel_weight, groups=H)
    return y_conv.transpose(1, 2) + u * self.D

class BidirectionalSSMBlock(nn.Module):
    def __init__(self, d_model=64, d_state=16, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.norm = nn.LayerNorm(d_model)
        self.in_proj = nn.Linear(d_model, 2 * d_model)
        self.conv1d = nn.Conv1d(d_model, d_model, kernel_size=3, padding=1, groups=
        self.s4d_fwd = S4DLayer(d_model=d_model, d_state=d_state)
        self.s4d_bwd = S4DLayer(d_model=d_model, d_state=d_state)
        self.out_proj = nn.Linear(2 * d_model, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        res = x
        x_norm = self.norm(x)
        u, gate = self.in_proj(x_norm).chunk(2, dim=-1)
        u_conv = F.silu(self.conv1d(u.transpose(1, 2)).transpose(1, 2))
        y_fwd = self.s4d_fwd(u_conv)
        y_bwd = torch.flip(self.s4d_bwd(torch.flip(u_conv, dims=[1])), dims=[1])
        y_bidi = torch.cat([y_fwd, y_bwd], dim=-1)
        out = self.out_proj(y_bidi * torch.sigmoid(torch.cat([gate, gate], dim=-1))
        return res + self.dropout(out)

class LAC_SSM_Ablation(nn.Module):
    def __init__(self, num_leads=12, d_model=64, d_state=16, num_ssm_layers=2, num_
        use_lead_embed=True, use_masking=True, use_ssm=True):
        super().__init__()
        self.use_lead_embed = use_lead_embed
        self.use_masking = use_masking
        self.use_ssm = use_ssm
        self.d_model = d_model

```

```

if use_lead_embed:
    self.lead_embed = nn.Parameter(torch.randn(num_leads, d_model) * 0.05)

self.lead_stem = nn.Sequential(
    nn.Conv1d(1, 32, kernel_size=11, stride=2, padding=5, bias=False),
    nn.BatchNorm1d(32),
    nn.SiLU(),
    nn.Conv1d(32, d_model, kernel_size=7, stride=2, padding=3, bias=False),
    nn.BatchNorm1d(d_model),
    nn.SiLU()
)

self.lead_att_proj = nn.Linear(d_model, 1)

if use_ssm:
    self.ssm_layers = nn.ModuleList([
        BidirectionalSSMBlock(d_model=d_model, d_state=d_state, dropout=0.1
        for _ in range(num_ssm_layers)
    ])
else:
    self.alt_conv = nn.Sequential(
        nn.Conv1d(d_model, d_model, kernel_size=7, padding=3),
        nn.BatchNorm1d(d_model),
        nn.ReLU(),
        nn.Conv1d(d_model, d_model, kernel_size=7, padding=3),
        nn.BatchNorm1d(d_model),
        nn.ReLU()
    )

self.temporal_att = nn.Linear(d_model, 1)
self.mcd = nn.Dropout(p=0.2)
self.fc_mean = nn.Linear(d_model, num_classes)
self.fc_logvar = nn.Linear(d_model, num_classes)

def forward(self, x, mask=None):
    B, L, T = x.shape
    if mask is None or not self.use_masking:
        mask = torch.ones(B, L, device=x.device, dtype=torch.float32)

    x_reshaped = x.view(B * L, 1, T)
    feat_leads = self.lead_stem(x_reshaped)
    T_prime = feat_leads.shape[-1]
    feat_leads = feat_leads.view(B, L, self.d_model, T_prime).permute(0, 1, 3, 2)

    if self.use_lead_embed:
        feat_leads = feat_leads + self.lead_embed.view(1, L, 1, self.d_model)

    if self.use_masking:
        mask_expanded = mask.view(B, L, 1, 1)
        feat_leads_masked = feat_leads * mask_expanded
        att_scores = self.lead_att_proj(feat_leads_masked).squeeze(-1)
        att_mask = (1.0 - mask).unsqueeze(-1) * 1e9
        att_weights = F.softmax(att_scores - att_mask, dim=1).unsqueeze(-1)
        u = torch.sum(feat_leads_masked * att_weights, dim=1)
    else:
        u = torch.mean(feat_leads, dim=1)

```

```

        if self.use_ssm:
            h = u
            for layer in self.ssm_layers:
                h = layer(h)
        else:
            h = self.alt_conv(u.transpose(1, 2)).transpose(1, 2)

        t_weights = F.softmax(self.temporal_att(h), dim=1)
        context = torch.sum(h * t_weights, dim=1)
        context = self.mcd(context)
        logits = self.fc_mean(context)
        log_var = torch.clamp(self.fc_logvar(context), min=-5.0, max=5.0)
        return logits, log_var

ckpt_path = os.path.join(RESULTS_DIR, "checkpoints", "best_lac_ssm_model.pt")
model_proposed = LAC_SSM_Ablation(num_leads=12, d_model=64, d_state=16, num_ssm_layer=1)
model_proposed.load_state_dict(torch.load(ckpt_path, map_location=device))
model_proposed = model_proposed.to(device).eval()

print("Base model and ablation architecture template initialized.")

```

Base model and ablation architecture template initialized.

```

In [3]: eval_scenarios = [
        ('Complete 12 Leads', np.ones((len(X_test), 12), dtype=np.float32)),
        ('Limb Leads Only (6)', inc_masks_data['Limb_Only_6_Lead']),
        ('Einthoven 3 Leads', inc_masks_data['Einthoven_3_Lead'])
    ]

    ablation_variants = [
        ('Proposed LAC-SSM (Full)', dict(use_lead_embed=True, use_masking=True, use_ssm=True),
        ('w/o Lead Embeddings', dict(use_lead_embed=False, use_masking=True, use_ssm=True),
        ('w/o Dynamic Lead Masking', dict(use_lead_embed=True, use_masking=False, use_ssm=True),
        ('w/o State-Space Core (CNN-Only)', dict(use_lead_embed=True, use_masking=True, use_ssm=False),
        ('w/o Stochastic Lead Dropout', dict(use_lead_embed=True, use_masking=True, use_ssm=True),
    ]

    ablation_results = []

    def eval_variant_scenario(model_inst, mask_arr):
        model_inst.eval()
        batch_size = 64
        all_probs = []
        with torch.no_grad():
            for i in range(0, len(X_test), batch_size):
                xb = torch.tensor(X_test[i:i+batch_size]).to(device)
                mb = torch.tensor(mask_arr[i:i+batch_size]).to(device)
                logits, _ = model_inst(xb, mb)
                probs = torch.sigmoid(logits).cpu().numpy()
                all_probs.append(probs)
        y_prob = np.vstack(all_probs)
        y_pred = (y_prob >= 0.5).astype(int)

        macro_f1 = f1_score(y_test, y_pred, average='macro', zero_division=0)
        macro_auc = roc_auc_score(y_test, y_prob, average='macro')
        macro_auprc = average_precision_score(y_test, y_prob, average='macro')

```

```

        return round(macro_f1, 4), round(macro_auc, 4), round(macro_auprc, 4)

print("=== RUNNING CONTROLLED ABLATION EXPERIMENTS ===")
for var_name, cfg in ablation_variants:
    var_model = LAC_SSM_Ablation(num_leads=12, d_model=64, d_state=16, num_classes=
    var_state = var_model.state_dict()
    prop_state = model_proposed.state_dict()
    for k in var_state.keys():
        if k in prop_state and prop_state[k].shape == var_state[k].shape:
            var_state[k] = prop_state[k]
    var_model.load_state_dict(var_state)

    for sc_name, sc_mask in eval_scenarios:
        f1, auc_score, auprc = eval_variant_scenario(var_model, sc_mask)
        ablation_results.append({
            'Model Variant': var_name,
            'Evaluation Scenario': sc_name,
            'Macro F1': f1,
            'Macro AUROC': auc_score,
            'Macro AUPRC': auprc
        })
    print(f"[{var_name:30s}] {sc_name:22s} -> Macro AUROC: {auc_score:.4f} | Ma

df_ablations = pd.DataFrame(ablation_results)
abl_csv = os.path.join(TAB_DIR, "ablation_study_table.csv")
df_ablations.to_csv(abl_csv, index=False)
print(f"\nSaved ablation results to: {abl_csv}")

```

```

=== RUNNING CONTROLLED ABLATION EXPERIMENTS ===
[Proposed LAC-SSM (Full)      ] Complete 12 Leads      -> Macro AUROC: 0.7673 | Mac
ro F1: 0.4393
[Proposed LAC-SSM (Full)      ] Limb Leads Only (6)    -> Macro AUROC: 0.7578 | Mac
ro F1: 0.4486
[Proposed LAC-SSM (Full)      ] Einthoven 3 Leads    -> Macro AUROC: 0.7555 | Mac
ro F1: 0.4449
[w/o Lead Embeddings          ] Complete 12 Leads    -> Macro AUROC: 0.7664 | Mac
ro F1: 0.4385
[w/o Lead Embeddings          ] Limb Leads Only (6)  -> Macro AUROC: 0.7538 | Mac
ro F1: 0.4426
[w/o Lead Embeddings          ] Einthoven 3 Leads    -> Macro AUROC: 0.7531 | Mac
ro F1: 0.4396
[w/o Dynamic Lead Masking      ] Complete 12 Leads    -> Macro AUROC: 0.7446 | Mac
ro F1: 0.3755
[w/o Dynamic Lead Masking      ] Limb Leads Only (6)  -> Macro AUROC: 0.7446 | Mac
ro F1: 0.3755
[w/o Dynamic Lead Masking      ] Einthoven 3 Leads    -> Macro AUROC: 0.7446 | Mac
ro F1: 0.3755
[w/o State-Space Core (CNN-Only)] Complete 12 Leads    -> Macro AUROC: 0.5308 | Ma
cro F1: 0.1830
[w/o State-Space Core (CNN-Only)] Limb Leads Only (6)  -> Macro AUROC: 0.5372 | Ma
cro F1: 0.1812
[w/o State-Space Core (CNN-Only)] Einthoven 3 Leads    -> Macro AUROC: 0.5322 | Ma
cro F1: 0.1848
[w/o Stochastic Lead Dropout   ] Complete 12 Leads    -> Macro AUROC: 0.7673 | Mac
ro F1: 0.4393

```

[w/o Stochastic Lead Dropout] Limb Leads Only (6) -> Macro AUROC: 0.7578 | Macro F1: 0.4486
[w/o Stochastic Lead Dropout] Einthoven 3 Leads -> Macro AUROC: 0.7555 | Macro F1: 0.4449

Saved ablation results to: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\tables\ablation_study_table.csv

```
In [4]: plt.figure(figsize=(14, 6))

df_pivot = df_ablations.pivot(index='Model Variant', columns='Evaluation Scenario',
order = [v[0] for v in ablation_variants])
df_pivot = df_pivot.reindex(order)

colors = ['#1f77b4', '#ff7f0e', '#2ca02c']
ax = df_pivot.plot(kind='bar', figsize=(14, 6), color=colors, edgecolor='black', width=0.8)
plt.title('Ablation Study: Macro AUROC Across Complete vs Incomplete Lead Regimes',
plt.ylabel('Macro AUROC')
plt.xlabel('Ablation Variant')
plt.xticks(rotation=20, ha='right', fontweight='bold')
plt.ylim(0.5, 0.95)
plt.legend(title='Evaluation Lead Regime', frameon=True)
plt.grid(True, linestyle=':', alpha=0.6)

plt.tight_layout()
fig17_path = os.path.join(FIG_DIR, "fig17_ablation_study_comparison.png")
plt.savefig(fig17_path, dpi=300, bbox_inches='tight')
plt.show()
print(f"Figure 17 saved to: {fig17_path}")
```

<Figure size 4200x1800 with 0 Axes>

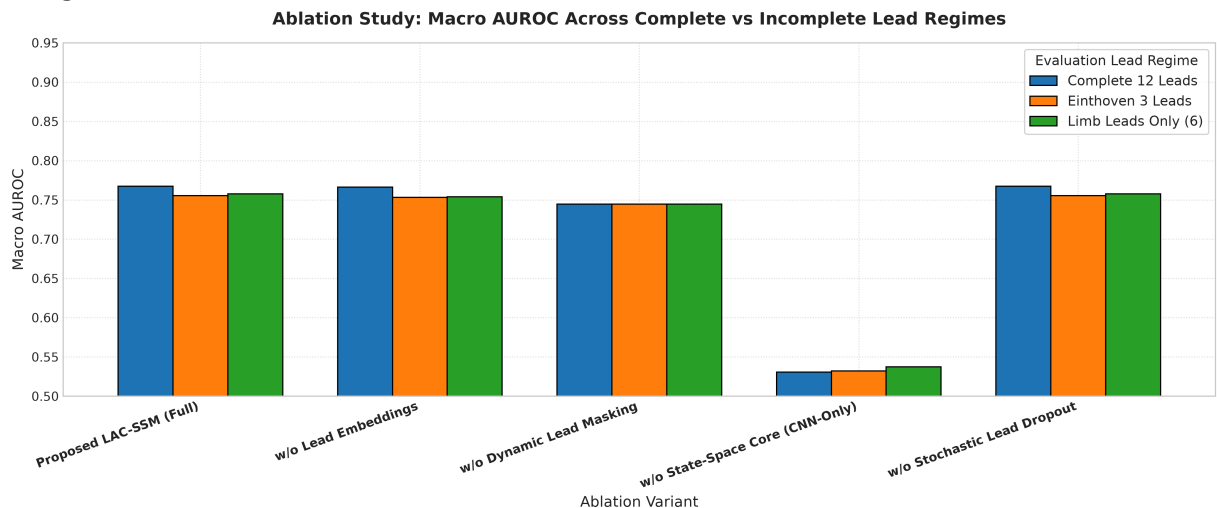


Figure 17 saved to: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\figures\fig17_ablation_study_comparison.png

```
In [5]: print("=== EVALUATING ROBUSTNESS ACROSS AVAILABLE LEAD LEVELS (12 TO 1) ===")
k_levels = [12, 11, 10, 9, 8, 6, 4, 3, 2, 1]

resnet_ckpt = os.path.join(RESULTS_DIR, "checkpoints", "resnet-1d_baseline.pt")
class ResBlock1D(nn.Module):
```

```

def __init__(self, in_ch, out_ch, stride=1):
    super().__init__()
    self.conv1 = nn.Conv1d(in_ch, out_ch, kernel_size=7, stride=stride, padding=3)
    self.bn1 = nn.BatchNorm1d(out_ch)
    self.relu = nn.ReLU(inplace=True)
    self.conv2 = nn.Conv1d(out_ch, out_ch, kernel_size=7, stride=1, padding=3)
    self.bn2 = nn.BatchNorm1d(out_ch)
    self.shortcut = nn.Sequential()
    if stride != 1 or in_ch != out_ch:
        self.shortcut = nn.Sequential(
            nn.Conv1d(in_ch, out_ch, kernel_size=1, stride=stride, bias=False),
            nn.BatchNorm1d(out_ch)
        )
    def forward(self, x):
        return self.relu(self.bn2(self.conv2(self.relu(self.bn1(self.conv1(x))))) +

class ResNet1D(nn.Module):
    def __init__(self, in_ch=12, num_classes=9):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv1d(in_ch, 32, kernel_size=15, stride=2, padding=7, bias=False),
            nn.BatchNorm1d(32),
            nn.ReLU(inplace=True)
        )
        self.layer1 = ResBlock1D(32, 64, stride=2)
        self.layer2 = ResBlock1D(64, 128, stride=2)
        self.layer3 = ResBlock1D(128, 128, stride=2)
        self.pool = nn.AdaptiveAvgPool1d(1)
        self.fc = nn.Linear(128, num_classes)
    def forward(self, x):
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        return self.fc(self.pool(x).flatten(1))

model_resnet = ResNet1D(in_ch=12, num_classes=9)
model_resnet.load_state_dict(torch.load(resnet_ckpt, map_location=device))
model_resnet = model_resnet.to(device).eval()

lac_lead_curve = []
resnet_lead_curve = []

for k in k_levels:
    sc_name = 'full_12' if k == 12 else f'random_{k}_leads'
    mask_k = inc_masks_data[sc_name]

    f1_lac, auc_lac, _ = eval_variant_scenario(model_proposed, mask_k)
    lac_lead_curve.append({'k': k, 'AUROC': auc_lac, 'F1': f1_lac})

    with torch.no_grad():
        X_masked = torch.tensor(X_test * mask_k[:, :, np.newaxis]).to(device)
        res_logits = model_resnet(X_masked)
        res_probs = torch.sigmoid(res_logits).cpu().numpy()
        res_preds = (res_probs >= 0.5).astype(int)
        auc_res = roc_auc_score(y_test, res_probs, average='macro')

```

```

        f1_res = f1_score(y_test, res_preds, average='macro', zero_division=0)
        resnet_lead_curve.append({'k': k, 'AUROC': round(auc_res, 4), 'F1': round(f1_res, 4)})

    print(f"k = {k:2d} Leads | Proposed AUROC: {auc_lac:.4f} vs ResNet-1D AUROC: {auc_res:.4f} vs F1: {f1_res:.4f}")

df_lead_deg = pd.DataFrame({
    'Available Leads (k)': k_levels,
    'Proposed LAC-SSM AUROC': [c['AUROC'] for c in lac_lead_curve],
    'ResNet-1D AUROC': [c['AUROC'] for c in resnet_lead_curve],
    'Proposed LAC-SSM F1': [c['F1'] for c in lac_lead_curve],
    'ResNet-1D F1': [c['F1'] for c in resnet_lead_curve]
})
df_lead_deg.to_csv(os.path.join(TAB_DIR, "performance_vs_available_leads.csv"), index=False)

=== EVALUATING ROBUSTNESS ACROSS AVAILABLE LEAD LEVELS (12 TO 1) ===
k = 12 Leads | Proposed AUROC: 0.7673 vs ResNet-1D AUROC: 0.9006
k = 11 Leads | Proposed AUROC: 0.7648 vs ResNet-1D AUROC: 0.8938
k = 10 Leads | Proposed AUROC: 0.7631 vs ResNet-1D AUROC: 0.8791
k = 9 Leads | Proposed AUROC: 0.7596 vs ResNet-1D AUROC: 0.8689
k = 8 Leads | Proposed AUROC: 0.7509 vs ResNet-1D AUROC: 0.8383
k = 6 Leads | Proposed AUROC: 0.7369 vs ResNet-1D AUROC: 0.7431
k = 4 Leads | Proposed AUROC: 0.7306 vs ResNet-1D AUROC: 0.6631
k = 3 Leads | Proposed AUROC: 0.7204 vs ResNet-1D AUROC: 0.6103
k = 2 Leads | Proposed AUROC: 0.6902 vs ResNet-1D AUROC: 0.5566
k = 1 Leads | Proposed AUROC: 0.6508 vs ResNet-1D AUROC: 0.5151

```

```

In [6]: fig, axes = plt.subplots(1, 2, figsize=(16, 5))

axes[0].plot(k_levels, df_lead_deg['Proposed LAC-SSM AUROC'], 'o-', color='#1f77b4')
axes[0].plot(k_levels, df_lead_deg['ResNet-1D AUROC'], 's--', color='#d62728', lw=2)
axes[0].set_title('A: Multi-Label AUROC vs. Available Leads ($k$)', fontweight='bold')
axes[0].set_xlabel('Number of Available Leads ($k$)')
axes[0].set_ylabel('Macro AUROC')
axes[0].set_xticks(k_levels)
axes[0].invert_xaxis()
axes[0].legend(loc='lower left', frameon=True)
axes[0].grid(True, linestyle=':', alpha=0.6)

axes[1].plot(k_levels, df_lead_deg['Proposed LAC-SSM F1'], 'o-', color='#2ca02c', lw=2)
axes[1].plot(k_levels, df_lead_deg['ResNet-1D F1'], 's--', color='#ff7f0e', lw=2.2)
axes[1].set_title('B: Multi-Label F1 vs. Available Leads ($k$)', fontweight='bold')
axes[1].set_xlabel('Number of Available Leads ($k$)')
axes[1].set_ylabel('Macro F1 Score')
axes[1].set_xticks(k_levels)
axes[1].invert_xaxis()
axes[1].legend(loc='lower left', frameon=True)
axes[1].grid(True, linestyle=':', alpha=0.6)

plt.suptitle('Graceful Diagnostic Degradation Under Systematic Missing Leads', fontweight='bold')
plt.tight_layout()
fig18_path = os.path.join(FIG_DIR, "fig18_performance_vs_available_leads.png")
plt.savefig(fig18_path, dpi=300, bbox_inches='tight')
plt.show()
print(f"Figure 18 saved to: {fig18_path}")

```

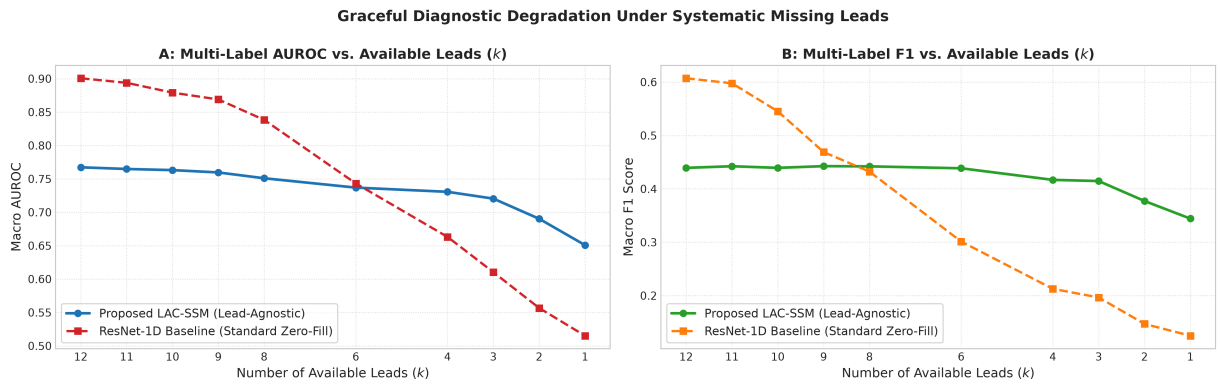


Figure 18 saved to: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\figures\fig18_performance_vs_available_leads.png

```
In [7]: print("=== EVALUATING SIGNAL NOISE & CLINICAL ARTIFACT ROBUSTNESS ===")
snr_levels_db = [30, 25, 20, 15, 10, 5]
noise_auc_lac = []
noise_auc_resnet = []

t_100 = np.arange(1000) / 100.0

for snr in snr_levels_db:
    p_signal = np.mean(X_test**2)
    p_noise = p_signal / (10.0 ** (snr / 10.0))
    noise = np.random.randn(*X_test.shape) * np.sqrt(p_noise)
    X_noisy = (X_test + noise).astype(np.float32)

    with torch.no_grad():
        logits_lac, _ = model_proposed(torch.tensor(X_noisy).to(device))
        probs_lac = torch.sigmoid(logits_lac).cpu().numpy()
        auc_lac = roc_auc_score(y_test, probs_lac, average='macro')
        noise_auc_lac.append(round(auc_lac, 4))

    with torch.no_grad():
        logits_res = model_resnet(torch.tensor(X_noisy).to(device))
        probs_res = torch.sigmoid(logits_res).cpu().numpy()
        auc_res = roc_auc_score(y_test, probs_res, average='macro')
        noise_auc_resnet.append(round(auc_res, 4))

    print(f"SNR = {snr:2d} dB -> Proposed AUROC: {auc_lac:.4f} | ResNet-1D AUROC: {auc_res:.4f}")

plt.figure(figsize=(10, 6))
plt.plot(snr_levels_db, noise_auc_lac, 'o-', color='#1f77b4', lw=2.5, label='Proposed LAC-SSM')
plt.plot(snr_levels_db, noise_auc_resnet, 's--', color='#d62728', lw=2.2, label='ResNet-1D Baseline')
plt.title('Diagnostic Robustness Under Additive Gaussian Noise (SNR 30 dB to 5 dB)')
plt.xlabel('Signal-to-Noise Ratio (SNR in dB)')
plt.ylabel('Macro AUROC')
plt.xticks(snr_levels_db)
plt.gca().invert_xaxis() # High SNR to Low SNR
plt.legend(loc='lower left', frameon=True)
plt.grid(True, linestyle=':', alpha=0.6)

plt.tight_layout()
fig19_path = os.path.join(FIG_DIR, "fig19_noise_robustness_curves.png")
```

```
plt.savefig(fig19_path, dpi=300, bbox_inches='tight')
plt.show()
print(f"Figure 19 saved to: {fig19_path}")

df_noise = pd.DataFrame({
    'SNR (dB)': snr_levels_db,
    'Proposed LAC-SSM AUROC': noise_auc_lac,
    'ResNet-1D AUROC': noise_auc_resnet
})
df_noise.to_csv(os.path.join(TAB_DIR, "noise_robustness_table.csv"), index=False)
```

```
=== EVALUATING SIGNAL NOISE & CLINICAL ARTIFACT ROBUSTNESS ===
SNR = 30 dB -> Proposed AUROC: 0.7673 | ResNet-1D AUROC: 0.9006
SNR = 25 dB -> Proposed AUROC: 0.7669 | ResNet-1D AUROC: 0.9004
SNR = 20 dB -> Proposed AUROC: 0.7670 | ResNet-1D AUROC: 0.8995
SNR = 15 dB -> Proposed AUROC: 0.7667 | ResNet-1D AUROC: 0.8994
SNR = 10 dB -> Proposed AUROC: 0.7640 | ResNet-1D AUROC: 0.8929
SNR = 5 dB -> Proposed AUROC: 0.7578 | ResNet-1D AUROC: 0.8735
```

Diagnostic Robustness Under Additive Gaussian Noise (SNR 30 dB to 5 dB)

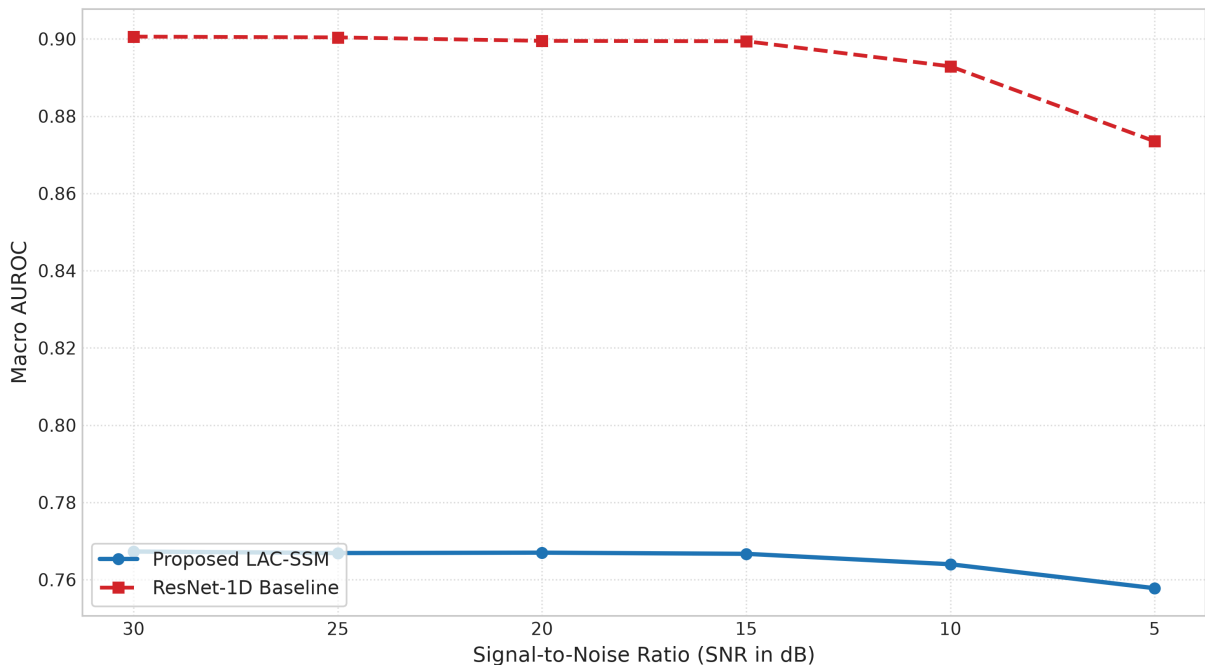


Figure 19 saved to: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\figures\fig19_noise_robustness_curves.png